Tarn

Architecture & Internals

Schema-first design, branded crypto types, strict TypeScript

May 2026

Contents

Tarn SDK — Implementation Architecture

Audience: developers evaluating Tarn for use, including the Bookish team for the migration off the pre-redesign client. **Companion docs:** `client/README.md` is the app-facing API reference; `docs/TARN_PROTOCOL.md` is the wire-protocol spec; `docs/SDK_REDESIGN.md` is the design doc this document describes the implementation of.

1. Summary

The Tarn SDK is a TypeScript library that gives apps typed CRUD over encrypted, user-owned data on Arweave. Apps declare collections and fields via `defineSchema()`; that declaration drives compile-time autocomplete, runtime validation, and the lifecycle namespaces hanging off the client. The crypto layer underneath uses branded types pervasively so primitives that are all "strings" at runtime — lookup keys, share keys, wrapped data keys, base64 vs. base64url vs. hex — cannot be confused at compile time. The SDK is built strict (`strict: true` plus `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`, and `noPropertyAccessFromIndexSignature`), with no `@ts-nocheck` anywhere in the source. Schemas are published to Arweave under `Type='app-schema'` so a future "always access your data" recovery client can decode entries without depending on the Tarn API at all — that recovery property is the structural reason for the schema-first design.

2. The shape apps see

```
import { TarnClient, defineSchema, TarnStorage } from 'tarn-client';

const schema = defineSchema({
  appId: 'your-app',
  version: 1,
  collections: {
    notes: {
      primaryKey: 'noteId',
      fields: {
        noteId:   'string',
        title:    'string',
        body:     'string?',
        priority: 'integer?',
      },
      shareable: true,
    },
  },
});

const tarn = await TarnClient.create({
  apiBase: 'https://api.tarn.dev',
  appId:   'your-app',
  schema,
  storage: TarnStorage.localStorage(),
});

await tarn.login(email, password);
await tarn.notes.create({ noteId: 'n1', title: 'Hello, Tarn' });
const all = await tarn.notes.list();
await tarn.notes.share(connection, 'n1');
```

`tarn` carries six top-level namespaces, plus one typed namespace per collection in the schema:

- `tarn.<collection>` — typed CRUD per collection (`create` , `update` , `get` , `list` , `delete`). Collections marked `shareable: true` also get `share` , `shareWithAll` , `unshare` , `listShared` . Updates are partial-merge: the SDK reads, merges the patch, re-validates the full record, and writes a chained entry. Apps work in primary-key space and never see Arweave txids.
- `tarn.connections` — the full connection lifecycle: `invite` (email handshake), `createInvite` / `previewInvite` / `redeemInvite` (link/QR token flow with non-consuming preview), `accept` , `list` / `listIncomingRequests` , `listIssuedInvites` / `revokeIssuedInvite` , `mute` / `unmute` , `setLabel` , `remove` . Connections are SDK objects (`{ share_pub, signing_pub, email?, label?, muted?, established_at?, initial_request_nonce? }`); apps pass them around without touching the underlying X25519 keys. Invite-flow returns are typed too — `InviteToken` , `InvitePreview` , `IssuedInvite` , `RedeemedInvite` , `IncomingRequest` .
- `tarn.account` — `changeCredentials(newEmail, newPassword, { phrase? })` , `delete()` . Routine credential rotation; `phrase` extends the recovery factor to the new

generation.

- `tarn.session` — `isLoggedIn()`, `clear()`, plus the multi-device server-side surface (`listDevices`, `revokeDevice`, `revokeAllOthers`, `revokeAll`).
- `tarn.recovery` — `export({ format: 'pdf' | 'json' })` to re-render the recovery kit. Rendering is purely client-side; Tarn never sees the phrase or the rendered bytes. Delivery to the user (download, print, app-operated transport) is the application's responsibility — the SDK deliberately exposes no transport surface for recovery material. Account recovery itself is on the top-level client (`tarn.recoverAccount({ phrase, newEmail, newPassword })`) since it's pre-auth.
- `tarn.advanced` — schema-less entry CRUD, raw blob fetch, direct share-log access. Power-user escape hatches; most apps never reach for these.

Lifecycle methods that don't fit a noun namespace stay on the top-level client: `login`, `register`, `recoverAccount`. Session persistence is automatic — `TarnClient.create()` reads the configured storage adapter and rehydrates a logged-in client when a valid blob is present, so PWAs and re-opened tabs come back already authenticated. `tarn.session.clear()` (or any logout-equivalent path) wipes the persisted blob.

3. What the type system is doing

This is the meat of the redesign — the layer that makes the schema-first surface ergonomic.

Schema-first with compile-time inference

`defineSchema()` is generic over its argument and uses `<const S>` capture so the literal shape of the input flows into the type system:

```
export function defineSchema<const S>(input: S): Schema<S extends SchemaInput ? S :
    never>
```

The input is intentionally not constrained at the parameter level — constraining to `SchemaInput` would widen literal types like `'string'` to `string`, which would defeat downstream record-type derivation. Instead, the constraint moves into the return type, runtime validation enforces structural validity at module load, and the `<const>` capture preserves the literal shape so `tarn.notes.create({ ... })` autocompletes from the schema and rejects unknown fields and type mismatches.

```
const tarn = await TarnClient.create({ schema: mySchema, /* ... */ });

await tarn.notes.create({
  noteId: 'n1',
  title:  'Foo',
  // titel: 'typo'    // ✗ TS error: unknown field
});

const note = await tarn.notes.get('n1');
//    ^? { noteId: string; title: string; body?: string; priority?: number; ... }
```

Plain JavaScript callers get the same runtime validation; only the autocomplete and the type-error surface are compile-time.

Branded types in the crypto layer

The crypto layer carries a fleet of distinct-but-string-shaped values. Branded types keep them apart at compile time:

```
declare const __brand: unique symbol;
type Brand<T, B extends string> = T & { readonly [__brand]: B };

export type LookupKey       = Brand<string, 'LookupKey'>;        // hex, 32 bytes
export type ShareKey        = Brand<string, 'ShareKey'>;         // base64url, 32 raw
    bytes
export type WrappedDataKey  = Brand<string, 'WrappedDataKey'>;   // v1 multi-factor
    envelope JSON
export type Base64          = Brand<string, 'Base64'>;
export type Base64Url       = Brand<string, 'Base64Url'>;
export type Hex             = Brand<string, 'Hex'>;
```

Constructors live next to format validators (`asLookupKey` , `asBase64Url` , etc.) and throw on malformed input. The brands erase to plain strings at runtime — no overhead — but a function that takes a `LookupKey` will not accept a `Base64` , and vice versa. The protocol layer is exactly the layer where mixing those up costs the most.

Strict-mode profile

```
{
  "compilerOptions": {
    "strict": true,
    "noUncheckedIndexedAccess": true,
    "exactOptionalPropertyTypes": true,
    "noImplicitOverride": true,
    "noPropertyAccessFromIndexSignature": true,
    "noFallthroughCasesInSwitch": true,
    "noImplicitReturns": true,
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "lib": ["ES2022", "DOM"]
  }
}
```

`noUncheckedIndexedAccess` flags `array[i]` as `T | undefined` rather than `T` , which catches off-by-one and missing-key bugs at compile time. `exactOptionalPropertyTypes` distinguishes "absent property" from "property explicitly set to `undefined` " — useful when wire-format objects (e.g., the wrapped-data-key envelope's optional fields, individual `Connection` fields) need their missing-vs-null distinction preserved on round-trip. `noPropertyAccessFromIndexSignature` forces `obj[key]` rather than `obj.key` on dynamically-keyed maps, which keeps the type system honest about which accesses are checked literals and which are user-supplied strings.

No `@ts-nocheck`

Every file in `client/src/` typechecks under the strict profile. There are no escape-hatch suppressions. The one place that could plausibly need them — WebCrypto's `BufferSource` boundary, where TypeScript's lib types disagree with the runtime contract — is handled by a single small helper:

```
// makes the boundary explicit: every WebCrypto input flows through `bs()`.
function bs(b: ArrayBufferView | ArrayBuffer): BufferSource {
  return b as BufferSource;
}
```

`bs()` is the only `as` cast that crosses that line. Every WebCrypto call (`crypto.subtle.sign`, `digest`, `wrapKey`, etc.) wraps its inputs in `bs()`. The convention is a one-line audit point rather than a sprinkling of suppressions.

Discriminated unions for protocol shapes

The share-log operation types are a tagged union:

```
type ShareLogOp =
  | { op: 'add';             content_id: ContentId; txid: Txid; share_key: ShareKey }
  | { op: 'update';          content_id: ContentId; txid: Txid }
  | { op: 'rotate';          content_id: ContentId; txid: Txid; share_key: ShareKey }
  | { op: 'remove';          content_id: ContentId }
  | { op: 'snapshot';        state: ShareLogSnapshot }
  | { op: 'rotate_identity'; new_share_pub: ShareKey; new_signing_pub: Base64; ... };
```

The discriminant (`op`) lets the consumer narrow exhaustively with `noFallthroughCasesInSwitch` catching forgotten cases. Same pattern for envelope versions (`v: 1 | 2 | 3 | 4`) on the wrapped-data-key envelope, and for collection-validation results (`{ ok: true, value: T } | { ok: false, errors: ValidationError[] }`).

4. Module layout

```
client/src/
  schema/                — DSL: defineSchema, validators, reserved-namespace enforcement
    define.ts, types.ts, validate.ts, reserved.ts, index.ts
  collections/           — typed CRUD wrapper over the protocol-layer client
    collection.ts, eid.ts, types.ts, index.ts
  sharing/               — Connection types and sharing helpers (public types)
    types.ts, index.ts
  storage/               — TarnStorageAdapter interface + memory/localStorage/custom
    built-ins
    adapter.ts, memory.ts, local-storage.ts, custom.ts, index.ts
  client/                — TarnClient class + lifecycle namespaces
    tarn-client.ts, types.ts, index.ts
    namespaces/
      connections.ts, account.ts, session.ts, recovery.ts, advanced.ts
  crypto.ts              — KDFs, key derivation, AES-KW wrap/unwrap, ECDSA helpers
  sharing.ts             — HPKE handshake, per-pair K_AB derivation
  share-log.ts           — append-only share-log read/write, op-type discriminated union
  recovery.ts            — BIP39 phrase generation, recovery KEK derivation, PDF
    rendering
  session-persistence.ts — at-rest serialize/resume + IndexedDB wrapping key
  tarn.ts                — protocol-layer client (the "underlying" — used internally by
    default)
  index.ts               — public barrel
```

The cut between `client/` (lifecycle namespaces over an injected protocol client) and `tarn.ts` (the protocol-layer client itself) is the redesign's core seam. `tarn.ts` is the converted-to-TS version of the original SDK — the wire-format speaker. The new code in `client/` is a thin typed layer that takes a schema and delegates to `tarn.ts` for everything that touches the network or the crypto material. `TarnClient.create()` constructs both halves and holds them together; tests inject a stub via the `underlying` factory option to drive the typed surface without standing up the full crypto stack.

`underlying` is optional (defaults to a fresh protocol-layer client). When supplied, `TarnClient.create()` skips session resume (the SDK can't resume an arbitrary stub-shaped client). When unsupplied — the production path — `create()` peeks at the storage adapter and, if a persisted blob is present, calls the legacy client's static `resumeSession()` to rehydrate the DEK chain, signing keys, and sharing keys before returning. Stale or corrupt blobs return null from resume and are cleared so the SDK doesn't retry on every page load.

5. Build pipeline

ESM-only. `esbuild` produces per-file ESM output, `tsc --emitDeclarationOnly` produces `.d.ts` files. No bundling — apps' bundlers tree-shake what they don't use:

```
client/dist/
  esm/        ESM with .js extensions, source maps
    schema/, collections/, sharing/, storage/, client/
    crypto.js, sharing.js, share-log.js, recovery.js, session-persistence.js, tarn.js
```

```
    index.js
  types/      .d.ts declaration files
    (mirror of src/)
```

The `package.json` `exports` map points to `dist/types/index.d.ts` for types and `dist/esm/index.js` for `import`. Source ships in the package too (`files: ['dist/esm/', 'dist/types/', 'src/', 'README.md']`) so consumers can step into TS source via source maps without unpacking the build.

CJS output was dropped during the single-envelope cleanup. Every supported runtime (Node 20+, modern browsers, Vite/Webpack/Rollup) handles ESM natively, and the `.js` ↔ `.cjs` extension juggling that ESM-source-emitted-as-CJS required (regex-rewriting `require("./X.js")` → `require("./X.cjs")` on every output file) was non-trivial code on the build path that bought nothing in practice.

`prepublishOnly: typecheck && test && build` gates publishes — a broken typecheck or test, or a missing dist/ entry, blocks the publish before anything reaches npm.

6. Testing layers

Four layers, each catching different things:

- **96 TypeScript unit tests** in `client/tests/{schema,collection,client}.test.ts`. Schema validation (reserved-namespace enforcement, optional-vs-required, enum membership), collection wrapping (the partial-merge update path, primary-key-to-Eid mapping, `share()` only existing on `shareable: true` collections), namespace delegation (each lifecycle method calls through to the right underlying method) via a `MockTarnClient` that satisfies `IUnderlyingClient`. Pure logic; no network.
- **400+ JavaScript unit tests** in `tests/unit/*.test.js`. Crypto primitives (HKDF derivation, AES-KW wrap/unwrap byte stability, P-256 retry rule, Argon2id determinism), per-content CEK handling, multi-factor wrapped-data-key envelope, share-log read/write under simulated Arweave, session persistence round-trips, recovery factor unwrap, invite-token redemption. These run via `tsx` so the JS test files import the in-progress `.ts` source modules directly.
- **Integration tests** against `wrangler dev` in `tests/test-*.mjs`. Full HTTP round-trip: register → app auth → set rules → login → write → read → share → recover → delete. Catches contract drift between SDK and API.
- **Four runnable examples** in `examples/`. Manually executed against a real API; catches end-to-end issues that mocks don't (real Arweave gateway responses, real Turbo upload latency, real CORS).
- **GitHub Actions CI** on every push to `dev` / `main`: typecheck + TS unit tests + JS unit tests + build + `npm pack --dry-run`. Green CI is the required precondition for any merge.

7. The recovery property

Schemas are published to Arweave under `Type='app-schema', App=<app_id>, V=<version>` via `tools/publish-schema.mjs`. Anyone with read access to Arweave (i.e., everyone, via a public gateway) can fetch the schema by app and version. Combined with the wrapped-data-key envelope being recoverable from `Type='cred'` blobs and content blobs being recoverable from `Type='entry'` blobs, this means: a "Tarn recovery" client could be built that reads everything it needs straight from Arweave gateways via GraphQL, with no Tarn API in the loop, given only the user's email + password (or recovery phrase) and the app id.

That client is on the roadmap, not shipped. The protocol-level enabling work — schema publication endpoint, fixed `Type` tag, deterministic envelope formats — is done. Apps inherit the property by virtue of using the SDK; nothing app-side needs to change to make it true.

This is the structural reason for the schema-first design. A wire-protocol-only client could never decode an app's records into their user-readable shape; the schema is what turns "decrypted opaque JSON" into "a record with named fields and types". Publishing it to the same permanent ledger as the data closes the recovery loop.

8. What stayed the same

The wire protocol is unchanged. `tarn.ts` (the protocol-layer client) was converted from JavaScript to TypeScript with no public-API changes — every byte that hits Arweave or the Tarn API is identical to what shipped before the redesign. The SDK redesign is API-shape-only at the SDK boundary. Nothing on Arweave, nothing in D1, no migration on either side.

This matters for the Bookish migration: Bookish does not need to re-encrypt or re-publish anything. The migration is a swap of the import statement and a rewrite of the call sites against the new typed surface. Existing data continues to decrypt under the new SDK because the new SDK reads the same envelopes it wrote.

9. Net new lines

The cumulative diff across the redesign is large — roughly 35,000 insertions across 155 files — but most of that is `.js` → `.ts` renames where Git counts every line of the renamed file as new. Real net new content is closer to 10,000 lines: the new TS modules (`schema/`, `collections/`, `client/`, `sharing/`, `storage/`), the schema publication endpoint and `tools/publish-schema.mjs`, the TS test suite, the four example apps, the build pipeline, the CI workflow, the rewritten README, and this document.

The redesign was scoped to one app to migrate (Bookish, the reference app). With one consumer and the protocol unchanged, the rip-and-replace was the right call: parallel APIs would have doubled the maintenance surface for the duration of the migration window, and the breaking-change cost is paid once.

10. What's not done

• **Bookish migration.** The pre-redesign SDK shape (`createEntry` / `getEntries` /etc. on a plain `TarnClient` instance) is still what Bookish depends on. The migration is the next deliverable; this document is the pitch artifact for it.

• **Recovery client.** The "always access your data" client described in §7 is on the roadmap. The protocol-level pieces are done; the client itself is a separate deliverable.

• `tarn.ts` **public-method types.** The protocol-layer client's public methods accept `: any` parameters in places where the typed surface above already constrains the inputs. Tightening those is a follow-up; the new typed surface is what apps see, and it's fully constrained.

• **Schema-evolution helpers.** `defineSchema({ ..., onUnknownField: 'strip' | 'preserve' | 'error' })` for opt-in laxer reads, and an opinion on whether a `migrate()` helper belongs in the SDK or in app code. The current SDK is strict-only ('error' equivalent); the field-evolution rules in `client/README.md` are the explicit answer for now. Apps write their own migration loops with `list()` + `update()` .